Introduction to Programming and Data Structures

Sorting, Searching

Malay Bhattacharyya

Associate Professor

MIU, CAIML, TIH
Indian Statistical Institute, Kolkata

November, 2023

1. Sorting

A Sorting Algorithm is used to rearrange a given array or list of elements according to a comparison operator on the elements.

2. Searching

3. Problems

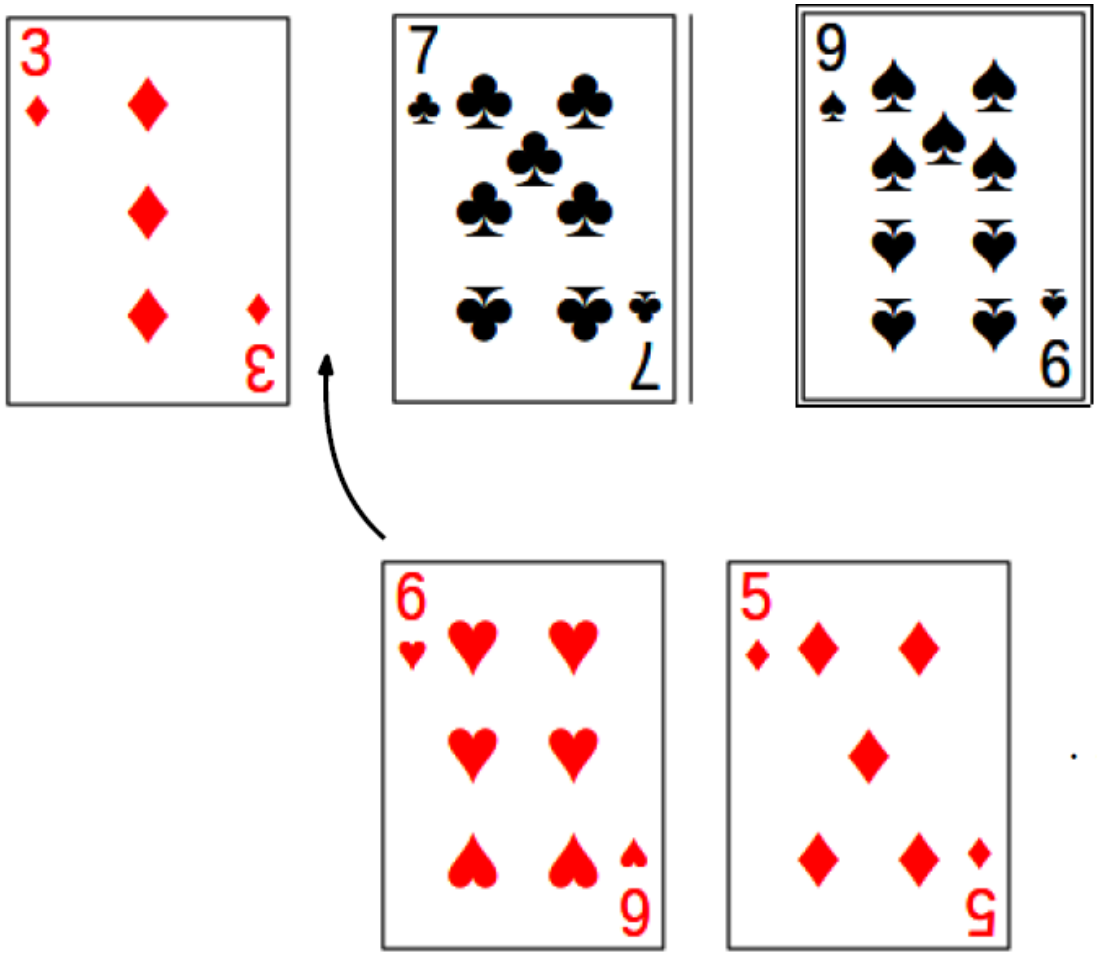
Bubble sort

```
bubbleSort(array)
  for i <- 1 to
indexOfLastUnsortedElement-1
    if leftElement > rightElement
      swap leftElement and
rightElement
end bubbleSort
```

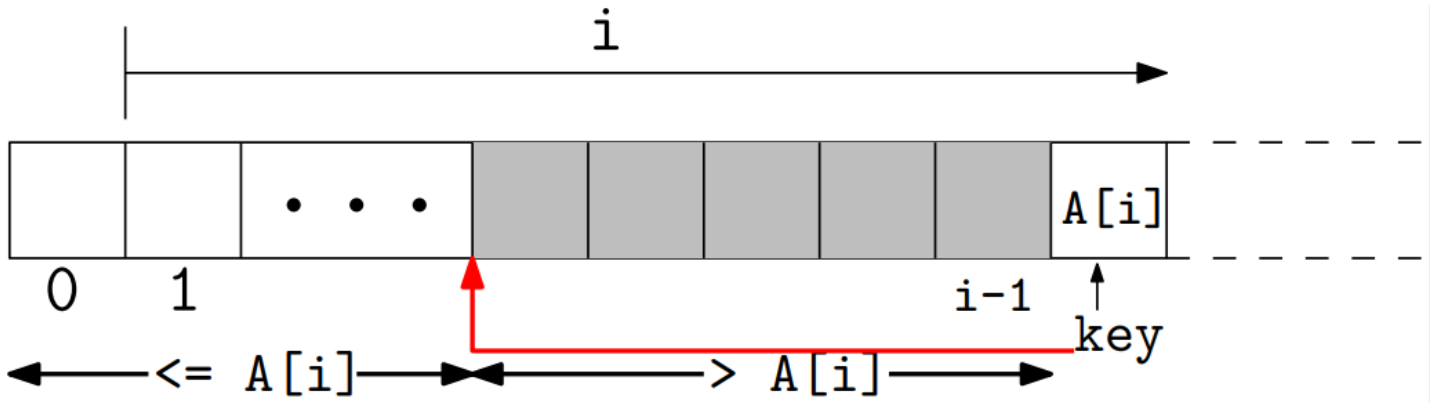
```
LIST = [2, 6, 4, 8, 1]
n = len(LIST)
for i in range(n-1):
    for j in range(n-i-1):
        if LIST[j+1] < LIST[j]:
            LIST[j], LIST[j+1] = LIST[j+1], LIST[j]
```

sorts an array by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. The algorithm iterates through the array multiple times, with each pass pushing the largest unsorted element to its correct position at the end.

Insertion sort



Insertion sort

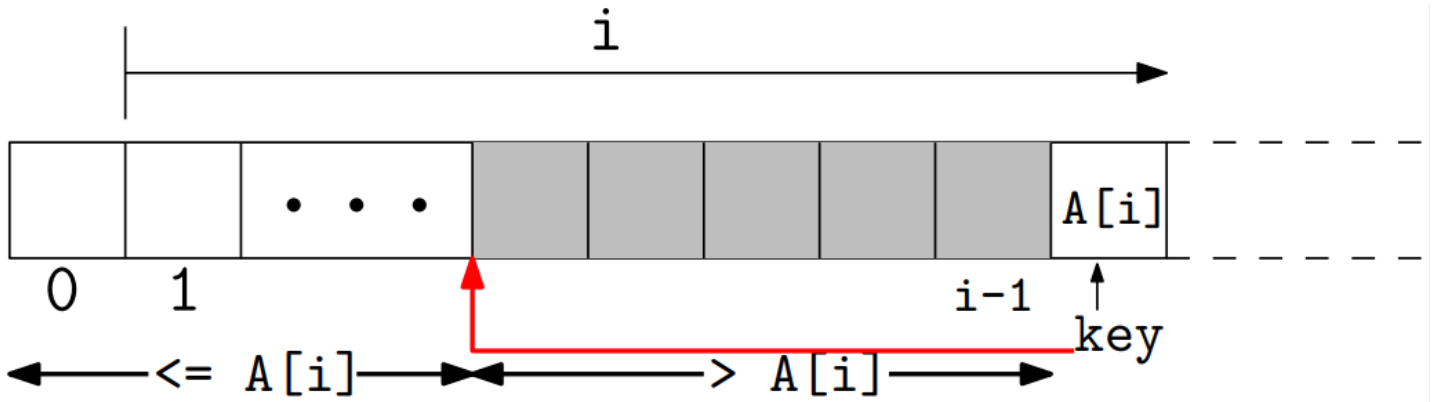


places an unsorted element at its suitable place in each iteration.

To sort an array of size N in ascending order iterate over the array and compare the current element (key) to its predecessor, if the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

```
insertionSort(array)
  mark first element as sorted
  for each unsorted element X
    'extract' the element X
    for j <- lastSortedIndex down to 0
      if current element j > X
        move sorted element to the right by 1
    break loop and insert X here
end insertionSort
```

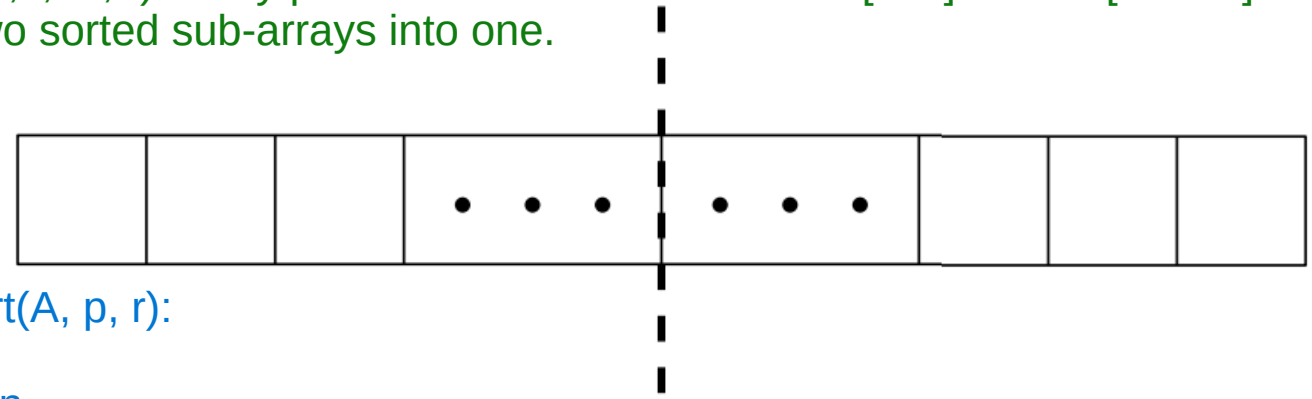
Insertion sort




```
LIST = [2, 6, 4, 8, 1]
n = len(LIST)
for i in range(1, n):
    key = LIST[i]
    # Find the right place for LIST[i] in LIST[0 ... i-1]
    j = i - 1
    while j >= 0 and LIST[j] > key:
        LIST[j+1] = LIST[j]
        j = j - 1
    LIST[j+1] = key
```

Merge sort

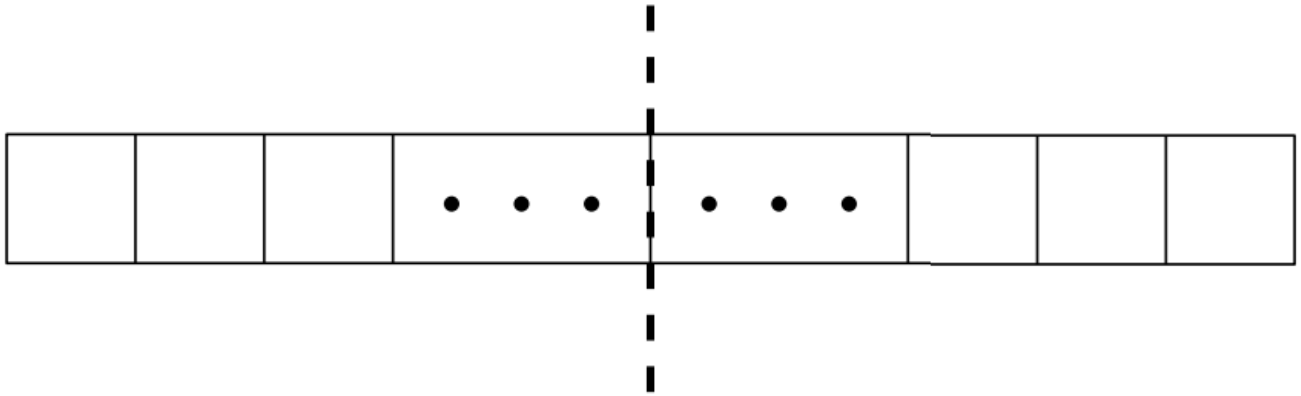
Divide and Conquer algorithm. It divides input array in two halves, calls itself for the two halves and then merges the two sorted halves. The merge() function is used for merging two halves. The merge(arr, l, m, r) is key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.



```
MergeSort(A, p, r):
    if p > r
        return
    q = (p+r)/2
    mergeSort(A, p, q)
    mergeSort(A, q+1, r)
    merge(A, p, q, r)
```

MERGE()
Have we reached the end of any of the arrays?
- No:
  - Compare current elements of both arrays
  - Copy smaller element into sorted array
  - Move pointer of element containing smaller element
- Yes:
  - Copy all remaining elements of non-empty array

Merge sort



```
def msort(LIST, beginning, end):
    if beginning < end:
        middle = (beginning + end) / 2
        msort(LIST, beginning, middle)
        msort(LIST, middle+1, end)
        merge(LIST, beginning, middle, end)
```

Merge sort

```
def merge(LIST, b, m, e):
    i, j, k = b, m + 1, 0
    while i <= m and j <= e:
        if LIST[i] < LIST[j]:
            AUXLIST[k] = LIST[i]
            k, i = k+1, i+1
        else if A[i] > A[j]:
            AUXLIST[k] = LIST[j]
            k, j = k+1, j+1
        else:
            AUXLIST[k] = LIST[i]
            k, i = k+1, i+1
            AUXLIST[k] = LIST[j]
            k, j = k+1, j+1
    while i <= m:
        AUXLIST[k] = LIST[i]
        k, i = k+1, i+1
    while j <= e:
        AUXLIST[k] = LIST[j]
```

Initialize an auxiliary list to store the merged elements
AUXLIST = [0] * (e - b + 1)

Compare elements from both halves and merge them into the auxiliary list

handle the case where one of the halves has remaining elements.

Copy any remaining elements from the first half

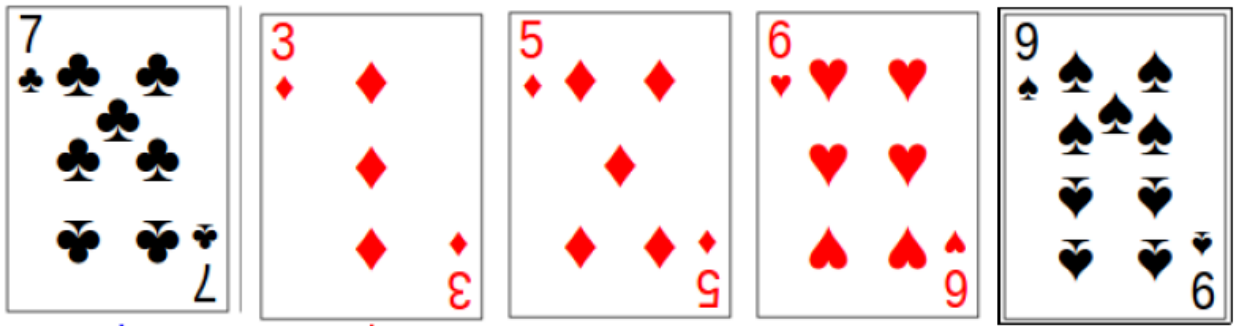
Copy any remaining elements from the second half

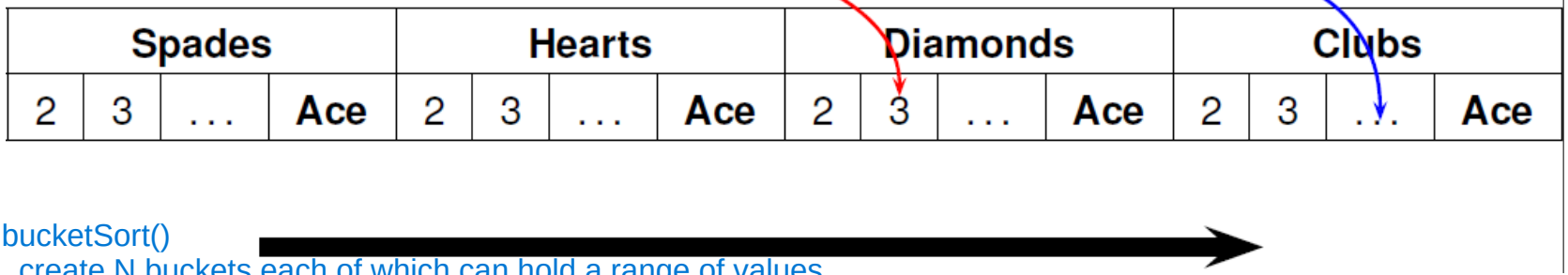
k, j = k + 1, j + 1
Copy the merged elements back to the original list
for x in range(e - b + 1):
LIST[b + x] = AUXLIST[x]

Bucket sort

involves dividing elements into various groups, or buckets. These buckets are formed by uniformly distributing the elements. Once the elements are divided into buckets, they can be sorted using any other sorting algorithm. Finally, the sorted elements are gathered together in an ordered fashion.

b[i] = floor((n)*arr[i])






```
bucketSort()
  create N buckets each of which can hold a range of values
  for all the buckets
    initialize each bucket with 0 values
  for all the buckets
    put elements into buckets matching the range
  for all the buckets
    sort elements in each bucket
  gather elements from each bucket
end bucketSort
```

Bucket sort

```
def bucketSort(array):
    bucket = []

    for i in range(len(array)):    # Create empty buckets
        bucket.append([])

    for j in array:                # Insert elements into their respective
        index_b = int(10 * j)      buckets
        bucket[index_b].append(j)

    for i in range(len(array)):    # Sort the elements of each
        bucket[i] = sorted(bucket[i])  bucket

    k = 0
    for i in range(len(array)):
        for j in range(len(bucket[i])):    # Get the sorted elements
            array[k] = bucket[i][j]
            k += 1
    return array
```

Timsort

hybrid sorting algorithm derived from merge sort and insertion sort.

Timsort is a highly optimized and stable version of merge sort that effectively combines insertion sort.

Timsort was implemented by Tim Peters in 2002 for use in the Python programming language.

- `https://en.wikipedia.org/wiki/Timsort`
- `https://realpython.com/sorting-algorithms-python/#pythons-built-in-sorting-algorithm`

Timsort

```
MIN= 32

def find_minrun(n):
    r = 0
    while n >= MIN:
        r |= n & 1
        n >>= 1
    return n + r
```

if N/minrun is a power of 2 as it will result in the best performance by merge sort.

Input: Unsorted collection
Output: Sorted collection

```
// Calculate run length
runLength := calculateRunLength(array);
// Perform insertion sort on each run
for start ← 0 to array.length by runLength do
    end := min(array.length − 1, start + runLength − 1);
    insertionSort(array, start, end);
end
```

sorting the runs

```
// Recursively merge adjacent runs
mergeSize := runLength;
while mergeSize < array.length do
    for left ← 0 to array.length by size ∗ 2 do
        mid := left + size − 1;
        right := min(array.length − 1, left + (2 ∗ size) − 1);
        if mid < right then
            merge(array, left, mid, right);
        end
    end
    mergeSize := mergeSize ∗ 2;
end
```

performing merge from merge sort

After each merge operation, we double the size of the run until it exceeds the length of the array.

The searching problem

Given a data structure S over a domain of data items D, verify (search) whether a given data item s belongs to D or not. If it belongs, then return the position (index) of s in D with respect to its organization in S.

Linear search

Let $LIST[] = \{30, 10, 20, 40, 50\}$ and $s = 40$.

Linear search is a sequential searching algorithm where we start from one end and check every element of the list until the desired element is found. It is the simplest searching algorithm.

```
def search(arr, x):

    for i in range(len(arr)):

        if arr[i] == x:
            return i

    return -1
```

Linear search

Let $LIST[] = \{30, 10, 20, 40, 50\}$ and $s = 40$.

$i = 0$	$\downarrow$					
Whether $LIST[i] = s$	30	10	20	40	50	FALSE

Linear search

Let $LIST[] = \{30, 10, 20, 40, 50\}$ and $s = 40$.

$i = 0$	↓					
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$		↓				
Whether $LIST[i] = s$	30	10	20	40	50	FALSE

Linear search

Let $LIST[] = \{30, 10, 20, 40, 50\}$ and $s = 40$.

$i = 0$	↓					
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$		↓				
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$			↓			
Whether $LIST[i] = s$	30	10	20	40	50	FALSE

Linear search

Let $LIST[] = \{30, 10, 20, 40, 50\}$ and $s = 40$.

$i = 0$	↓					
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$		↓				
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$			↓			
Whether $LIST[i] = s$	30	10	20	40	50	FALSE
$i = i + 1$				↓		
Whether $LIST[i] = s$	30	10	20	40	50	TRUE

Return $i = 3$.

Jump search

searching algorithm for sorted arrays. The basic idea is to check fewer elements (than linear search) by jumping ahead by fixed steps or skipping some elements in place of searching all elements.

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Further assume that the block size (of jump) is $b = 2$.

Define the value of k, the number of jump: Optimal jump size is N where the N is the length of array

Jump the array k-by-k searching by the condition Array[i] < valueWanted < Array[i+k] {if equality found then element found}

Do a linear search between Array[i] and Array[i + k]

```
import math
def jumpSearch(arr, x, n):
    # Finding block size to be jumped
    step = math.sqrt(n)
    # Finding the block where the element may be present (if it is present)
    prev = 0
    while arr[int(min(step, n) - 1)] < x:
        prev = step
        step += math.sqrt(n)
        if prev >= n:
            return -1  # Element is not present in the array
    # Doing a linear search for x in the block beginning with prev.
    while arr[int(prev)] < x:
        prev += 1
        # If we reached the next block or the end of the array, the element is not present.
        if prev == min(step, n):
            return -1  # Element is not present in the array
    # If the element is found
    if arr[int(prev)] == x:
        return int(prev)
    return -1  # Element is not present in the array
```

Jump search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Further assume that the block size (of jump) is $b = 2$.

$pre_i = 0, i = b$			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE

Why is n the optimal value for m in a jump search?

Let m be the jump size and n be the number of elements.

In the worst case, the maximum number of elements you have to check is the maximum number of jumps (n/m - 1) plus the number of elements between jumps (m), and the time you take is approximately proportional to the total number of elements you check.

The goal in choosing m, therefore, is to minimize: (n/m)+m-1.
derivative w.r.t to m to get, 1 - (n/m2), for minimum

1 - (n/m2) = 0
= > m = n

Jump search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Further assume that the block size (of jump) is $b = 2$.

$pre_i = 0, i = b$			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$pre_i = i, i = i + b$					↓	
Whether $LIST[i] < s$	10	20	30	40	50	FALSE

Jump search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Further assume that the block size (of jump) is $b = 2$.

$pre_i = 0, i = b$			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$pre_i = i, i = i + b$					↓	
Whether $LIST[i] < s$	10	20	30	40	50	FALSE
$pre_i = pre_i + 1$				↓		
Whether $LIST[pre_i] = s$	10	20	30	40	50	TRUE

Jump search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Further assume that the block size (of jump) is $b = 2$.

$pre_i = 0, i = b$			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$pre_i = i, i = i + b$					↓	
Whether $LIST[i] < s$	10	20	30	40	50	FALSE
$pre_i = pre_i + 1$				↓		
Whether $LIST[pre_i] = s$	10	20	30	40	50	TRUE

Note: The block size in jump search is taken as $b = \sqrt{n}$, where n denotes the size of the list.

Binary search

initializes pointers low and high to the array's start and end, respectively. In each iteration, it calculates the middle index, mid, and compares the element at mid with the target. Depending on the comparison, it narrows the search range by updating low or high. This process continues until the target is found or the pointers cross.

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

Binary search can be implemented only on a sorted list of items. If the elements are not sorted already, we need to sort them first.

```
Recursive Method
binarySearch(arr, x, low, high)
    if low > high
        return False
    else
        mid = (low + high) / 2
        if x == arr[mid]
            return mid
        else if x > arr[mid]        // x is on the right side
            return binarySearch(arr, x, mid + 1, high)
        else                         // x is on the left side
            return binarySearch(arr, x, low, mid - 1)
```

Binary search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

$l = 0, r = 4, i = l + (r - l)/2 = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE

Iteration Method
do until the pointers low and high meet each other.

```
mid = (low + high)/2
if (x == arr[mid])
    return mid
else if (x > arr[mid]) // x is on the right side
    low = mid + 1
else                   // x is on the left side
    high = mid - 1
```

Binary search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

$l = 0, r = 4, i = l + (r - l)/2 = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE
			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE

```
def binarySearch(arr, l, r, x):
    while l <= r:
        mid = l + (r - l) // 2
        # Check if x is present at mid
        if arr[mid] == x:
            return mid
        # If x is greater, ignore left half
        elif arr[mid] < x:
            l = mid + 1
        # If x is smaller, ignore right half
        else:
            r = mid - 1
    # not present
    return -1
```

Binary search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

$l = 0, r = 4, i = l + (r - l)/2 = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE
			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$l = i+1, i = l+(r-l)/2 = 3$				↓		
Whether $LIST[i] = s$	10	20	30	40	50	TRUE

Comparing linear/jump/binary search

	Linear search	**Jump search**	**Binary search**
Requirements	Nothing	Ordered list	Ordered list
Backward scan required	No	Once	Multiple times
Time complexity (best case)	$O(1)$	$O(b)$	$O(1)$
Time complexity (average case)	$O(n)$	$O(\sqrt{n})$	$O(\log n)$
Time complexity (worst case)	$O(n)$	$O(\sqrt{n})$	$O(\log n)$

Interpolation search

Given a sorted array of n uniformly distributed values arr[]

Binary Search always goes to the middle element to check. On the other hand, interpolation search may go to different locations according to the value of the key being searched.

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

// The idea of formula is to return higher value of pos
// when element to be searched is closer to arr[hi]. And
// smaller value when closer to arr[lo]

arr[] ==> Array where elements need to be searched
x ==> Element to be searched
lo ==> Starting index in arr[]
hi ==> Ending index in arr[]

pos = lo + [((x - arr[lo]) * (hi - lo)) / (arr[hi] - arr[lo])]

The formula for pos can be derived as follows.

Let's assume that the elements of the array are linearly distributed.
General equation of line : y = m*x + c.
y is the value in the array and x is its index.
Now putting value of lo,hi and x in the equation
arr[hi] = m*hi+c ----(1)
arr[lo] = m*lo+c ----(2)
x = m*pos + c ----(3)
m = (arr[hi] - arr[lo])/ (hi - lo)
subtracting eqxn (2) from (3)
x - arr[lo] = m * (pos - lo)
lo + (x - arr[lo])/m = pos
pos = lo + (x - arr[lo]) *(hi - lo)/(arr[hi] - arr[lo])

Interpolation search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

$i = 0 + (40 - 10) * (4 - 0)/(50 - 10) = 3$				↓		
Whether $LIST[i] = s$	10	20	30	40	50	TRUE

```
def interpolationSearch(arr, lo, hi, x):
    # Check if x is within the bounds of the array
    if lo <= hi and x >= arr[lo] and x <= arr[hi]:
        pos = lo + ((hi - lo) // (arr[hi] - arr[lo])) * (x - arr[lo])# Calculate the position using interpolation formula
        if arr[pos] == x:      # If the element is found at the calculated position
            return pos
        if arr[pos] < x:       # If x is greater, search in the right subarray
            return interpolationSearch(arr, pos + 1, hi, x)
        if arr[pos] > x:       # If x is smaller, search in the left subarray
            return interpolationSearch(arr, lo, pos - 1, x)
    return -1  # Element is not present in the array
```

Interpolation search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$.

$i = 0 + (40 - 10) * (4 - 0)/(50 - 10) = 3$				↓		
Whether $LIST[i] = s$	10	20	30	40	50	TRUE

Note: The interpolation search is efficient over binary search for the ordered lists having uniformly distributed data items.

Fibonacci search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Note that, the index of largest Fibonacci number that is greater than or equal to the length of given array is $f = 5$.

Works for sorted arrays; A Divide and Conquer Algorithm.;
Fibonacci Search divides given array into unequal parts unlike binary. ;
Binary Search uses a division operator to divide range. Fibonacci Search doesn't use /, but uses + and -.
The division operator may be costly on some CPUs.

we have the length of the given list. we find the smallest Fibonacci number greater than or equal to the size of the list. Let us say that this is the nth Fibonacci number. we find the (n-2)th Fibonacci number.

We use this as the index to divide the list into two parts. That is, we look at this index in the list, and assuming the list is sorted in increasing order, if the item at this index is smaller than the target, then we eliminate the left side, otherwise, we eliminate the right side. We do this until we find the item we are looking for, which will happen when the calculated index's item will match the target.

Fibonacci search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Note that, the index of largest Fibonacci number that is greater than or equal to the length of given array is $f = 5$.

$i = Fibonacci(f-2) = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE

```
def FibonacciSearch(arr, x):
    m = 0
    while FibonacciGenerator(m) < len(arr): #
        m = m + 1
    offset = -1
    while (FibonacciGenerator(m) > 1):
        i = min( offset + FibonacciGenerator(m - 2) , len(arr) - 1)
        print('Current Element : ',arr[i])
        if (x > arr[i]):
            m = m - 1
            offset = i                    eliminates values less than x
        elif (x < arr[i]):
            m = m - 2                     eliminates values greater than x
        else:
            return i
    if(FibonacciGenerator(m - 1) and arr[offset + 1] == x):    if there is one last element left
        return offset + 1
    return -1                             if not found
print('Number found at index : ',FibonacciSearch(arr, x))
```

```
def FibonacciGenerator(n):
    if n < 1:
        return 0
    elif n == 1:
        return 1
    else:
        return FibonacciGenerator(n - 1) +
FibonacciGenerator(n - 2)

arr = [10, 22, 30, 44, 56, 58, 60, 70, 100, 110, 130]
x = 60
```

Fibonacci search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Note that, the index of largest Fibonacci number that is greater than or equal to the length of given array is $f = 5$.

$i = Fibonacci(f-2) = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE
			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE

Fibonacci search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Note that, the index of largest Fibonacci number that is greater than or equal to the length of given array is $f = 5$.

$i = Fibonacci(f-2) = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE
			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$f = 4-2+1 = 3, i = i + Fibonacci(f-2) = 3$				↓		
Whether $LIST[i] = s$	10	20	30	40	50	TRUE

Fibonacci search

Let $LIST[] = \{10, 20, 30, 40, 50\}$ and $s = 40$. Note that, the index of largest Fibonacci number that is greater than or equal to the length of given array is $f = 5$.

$i = Fibonacci(f-2) = 2$			↓			
Whether $LIST[i] = s$	10	20	30	40	50	FALSE
			↓			
Whether $LIST[i] < s$	10	20	30	40	50	TRUE
$f = 4-2+1 = 3, i = i + Fibonacci(f-2) = 3$				↓		
Whether $LIST[i] = s$	10	20	30	40	50	TRUE

Note: Fibonacci search is a suitable replacement of binary search where the list is unbounded. Unlike binary search, it does not use the costly division operation (though avoidable with bitwise shift).

Problems

1. Let us define a sequence $a_0, a_1, a_2, \ldots, a_{n-1}$ as Λ-*bitonic* if there exists a j, $0 \leq j < n$, such that $a_0 < a_1 < \cdots < a_j > a_{j+1} > \cdots > a_{n-1}$. Consider an Λ-*bitonic* array consisting of integer entries. Write a program to efficiently find the largest element of the array.